概念

符号(单个字符) → 字母表(符号集, {0,1}) → 符号串(字母表符号组成的串(0100)) → 语言(Σ上符号串的集合) → 问题(判定性)

形式语言　形式文法

克林闭包: $\Sigma^* = \bigcup_{i=0}^{\infty} \Sigma^i$ → 一个字母表能组成的所有串

正闭包: $\Sigma^+ = \bigcup_{i=1}^{\infty} \Sigma^i$ → 一个字母表能组成的除ε外所有串

DFA

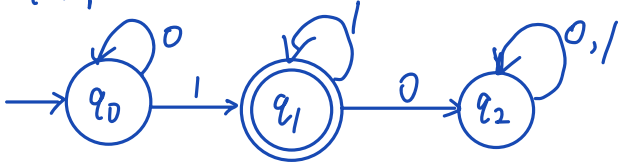
DFA $A=(Q, \Sigma, v, q_0, F)$

- Q: 有限状态集
- Σ: 有限符号集(字母表)
- v: 状态转移函数 $Q \times \Sigma \to Q$　($v(q_0, 1)=q_1$: q_0读入1转到q_1)
- $q_0 \in Q$: 初始状态
- $F \subseteq Q$: 接受状态(集)

DFA $A=(\{q_0, q_1, q_2\}, \{0,1\}, v, q_0, \{q_1\})$

$v=\{((q_0,0),q_0),((q_0,1),q_1),((q_1,0),q_2),((q_1,1),q_1),((q_2,0),q_2),((q_2,0),q_2)\}$

状态转移图:



状态转移表:

	0	1
→q_0	q_0	q_1
*q_1	q_2	q_1
q_2	q_2	q_2

完全形DFA: 每个状态对每个符号都有定义(补陷阱状态$q_\perp$)

- 陷阱状态: 一旦进入就不可能再接受
- 无效状态: 不可达, 可删除

最简形 DFA：无 无效状态

最小形 DFA：识别同一语言的DFA中状态数最少的

扩展转移函数 $\hat{\delta}(q_0, w)=q$：从 q_0 出发，读入串 w 到达状态 q

DFA定义的语言是正则语言：$L(M)=\{w\in\Sigma^* \mid \hat{\delta}(q_0, w)\in F\}$

DFA 设计：

① 含某子串：记录当前匹配到子串哪部分

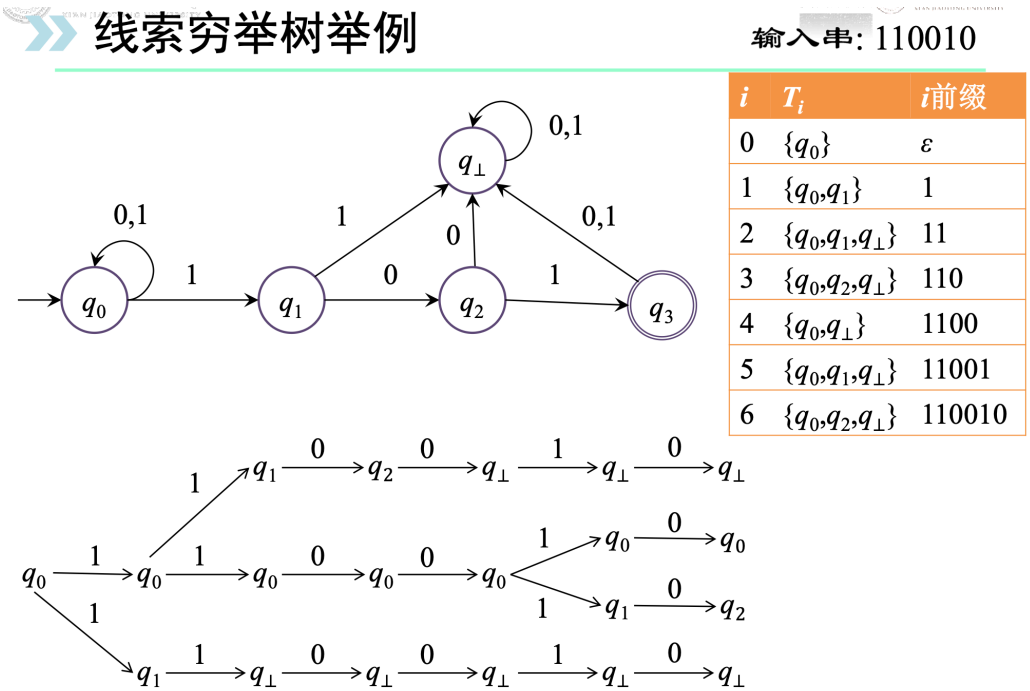
② 以某串结尾：记录当前已读前缀的最长相关后缀

③ 最多/至少/恰好n个某字符：记录出现次数，超过后进陷阱或稳定状态

④ 偶数个0，奇数个1：记录奇偶性组合

⑤ 二进制能被m整除：状态表示当前余数 $0\sim m-1$

线索穷举树：



NFA

NFA $A=(Q,\Sigma,v,q_0,F)$

- Q：有限状态集
- Σ：有限符号集（字母表）
- v：状态转移函数 $Q\times\Sigma\to 2^Q$ $(2^Q=\{S\mid S\subseteq Q\}$
- $q_0\in Q$：初始状态
- $F\subseteq Q$：接受状态（集）

扩展转移函数：$\tilde{v}(q,\varepsilon)=\{q\}$ $\tilde{v}(q,wa)=\bigcup_{p\in\tilde{v}(q,w)} v(p,a)$

↳始端为q，串为w的所有路径末端组成的集合

语言：$L(A)=\{w\in\Sigma^*\mid \tilde{v}(q_0,w)\cap F\neq\phi\}$ 有一个成功就接受

DFA与NFA等价：语言L被某个DFA接受 当且仅当它被某个NFA接受

NFA → DFA：子集构造法

NFA $N(Q,\Sigma,v,q_0,F)\to$ DFA $D(2^Q,\Sigma,v',\{q_0\},F')$

$F'=\{S\mid S\subseteq Q, S\cap F\neq\phi\}$

$v'(S,a)=\bigcup_{q\in S} v(q,a)$

从$\{q_0\}$开始，对每个状态集合S和输入符号a，计算后继集合，把新集合加入DFA状态，重复直到无新状态。含NFA接受状态的是DFA接受状态。（用状态转移表）

ε转移：不消耗输入符号发生状态转移

ε-NFA：含ε转移的NFA

ε闭包 $w(q)$：自身+经过连续ε转移所能到达的状态集

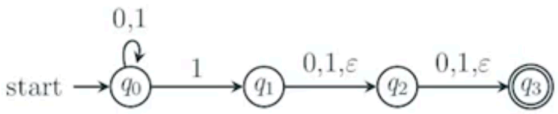
集合ε闭包 $w(S)$：$w(q)$的并　　ε-闭集：$S=w(S)$

ε-NFA的状态转移集合需要求ε-闭集　　$\hat{\delta}(q,\varepsilon)=w(q)$
$\hat{\delta}(q,xa)=w(r)$

ε-NFA语言：$L(B)=\{w\in\Sigma^* \mid \hat{\delta}(q_0,w)\cap F\neq\emptyset\}$

ε-NFA → DFA 的子集构造法：初态为 $w(q_0)$，对每个状态，根据ε-闭包算新状态

续例 11. 将下图 L 的 ε-NFA，转为等价的 DFA.



	0	1	ε	w( ) ~~ECLOSE( )~~
→ q_0	$\{q_0\}$	$\{q_0,q_1\}$	$\emptyset$	$\{q_0\}$
q_1	$\{q_2\}$	$\{q_2\}$	$\{q_2\}$	$\{q_1,q_2,q_3\}$
q_2	$\{q_3\}$	$\{q_3\}$	$\{q_3\}$	$\{q_2,q_3\}$
$*q_3$	$\emptyset$	$\emptyset$	$\emptyset$	$\{q_3\}$
→ $\{q_0\}$	$\{q_0\}$	$\{q_0,q_1,q_2,q_3\}$		
$*\{q_0,q_1,q_2,q_3\}$	$\{q_0,q_2,q_3\}$	$\{q_0,q_1,q_2,q_3\}$		
$*\{q_0,q_2,q_3\}$	$\{q_0,q_3\}$	$\{q_0,q_1,q_3,q_2\}$		
$*\{q_0,q_3\}$	$\{q_0\}$	$\{q_0,q_1,q_2,q_3\}$		

	0	1	ε	w( )
→ q_0	$\emptyset$	$\{q_1\}$	$\{q_1\}$	$\{q_0,q_1,q_2\}$
q_1	$\{q_0,q_1\}$	$\emptyset$	$\{q_2\}$	$\{q_1,q_2\}$
$*q_2$	$\emptyset$	$\emptyset$	$\emptyset$	$\{q_2\}$

	0	1
→ $*\{q_0,q_1,q_2\}$	$\{q_0,q_1,q_2\}$	$\{q_1,q_2\}$
$*\{q_1,q_2\}$	$\{q_0,q_1,q_2\}$	$\emptyset$

RE.

语言的运算：① 并：$L_1 \cup L_2 \rightarrow R+S$

② 连接：$L_1 L_2 = \{xy \mid x \in L_1, y \in L_2\} \rightarrow RS$

③ 克林闭包：$L^* = L^0 \cup L^1 \cup L^2 \cup \cdots \quad R^*$

$\Sigma^0 = \{\varepsilon\}$ $\phi^n = \phi$ $\phi^* = \{\varepsilon\}$ $\phi^0 = \{\varepsilon\}$ $\{\varepsilon\}^0 / \{\varepsilon\}^n / \{\varepsilon\}^* = \{\varepsilon\}$

正则表达式：$\phi \rightarrow$ 空语言，$\varepsilon \rightarrow \{\varepsilon\}$，$a \rightarrow \{a\}$，$r \rightarrow R$，$s \rightarrow S$

$r+s$，rs，r^*，$(r) \rightarrow R \cup S$，RS，R^*，R

$R \rightarrow \phi \mid \varepsilon \mid a \mid RR \mid R^* \mid R+R \mid (R)$

语言是串的集合

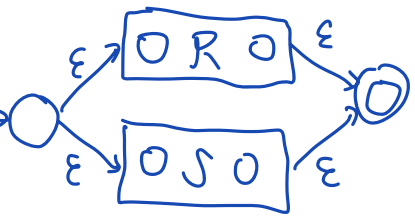
例：不含两个0的语言：$(1+01)^*(0+\varepsilon)$ / $1^*(011^*)^*(0+\varepsilon)$

偶数个0：$(1^*01^*01^*)^*$ 奇数个1：$(0^*10^*)^* + (0^*10^*10^*)^*$

DFA $\Leftrightarrow$ ε-NFA

RE

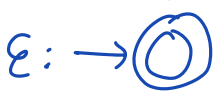
RE $\rightarrow$ ε-NFA：① $r+s$：



ϕ：



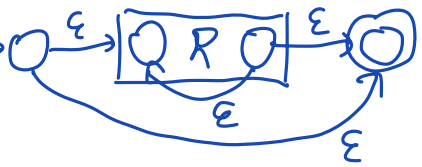
ε：



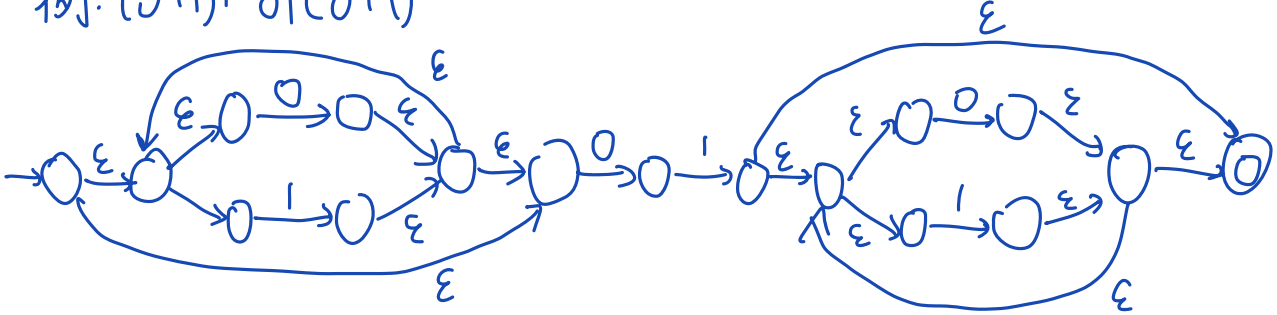
② rs：



③ r^*：



例：$(0+1)^*01(0+1)^*$

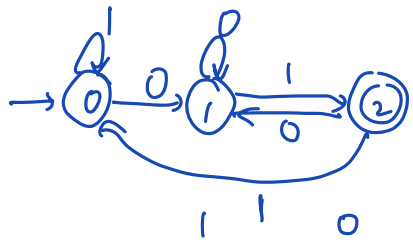


DFA → RE:

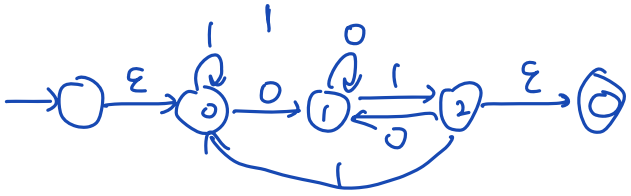
g-NFA: ①只有一个接受状态 ②没有射入到初始状态的弧

③没有从接受状态射出的弧 ④弧上标记为正则表达式

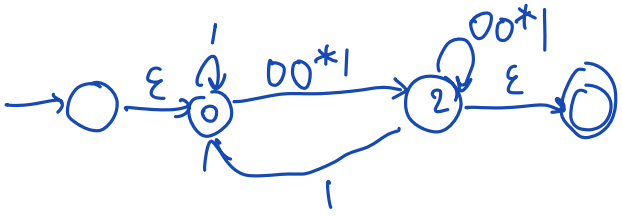
NFA 状态化简:



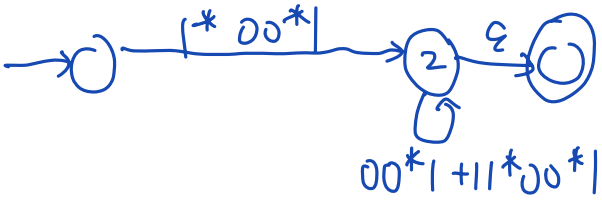
① 构造 g-NFA:



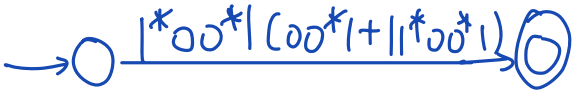
② 消除 1:



③ 消除 0:



④ 消除 2:



RL

正则语言的封闭性：(连接、并、闭包、交、补、差、逆、同态、逆同态)
运算结果依然属于该语言簇

泵引理：如果语言 L是正则的，∃正整数N，∀w∈L，只要 |w|≥N，
可以将 w分为三部分 w=xyz，满足：

① $y \neq \varepsilon$ ($|y|>0$)

② $|xy| < N$

③ $\forall k \geq 0$，$xy^kz \in L$ → 通过 y 泵出新串

满足泵引理不一定正则，但正则语言都满足泵引理

例：$\{0^m 1^m | m>1\}$ 不是正则语言：

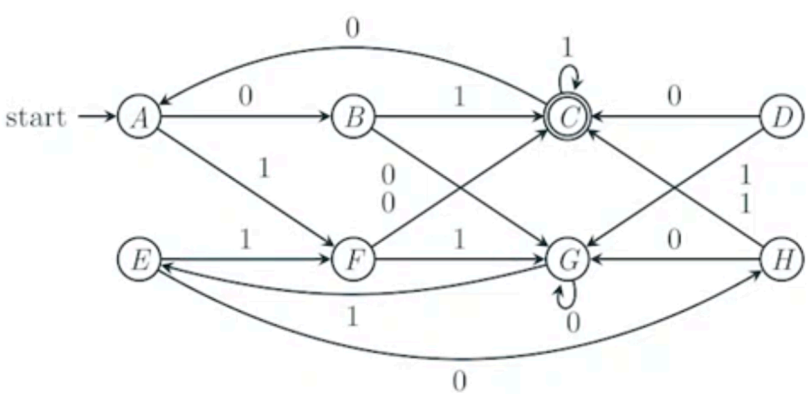
$w = 0^n 1^n$，把0分成三段，取中间一段，0就会和1数量不同

即 $x = 0^{n-k}$，$y = 0^k$，$z = 0^{n-m} 1^n$，则 $xy^iz = 0^{n-k} 1^n$ ($i=0$)

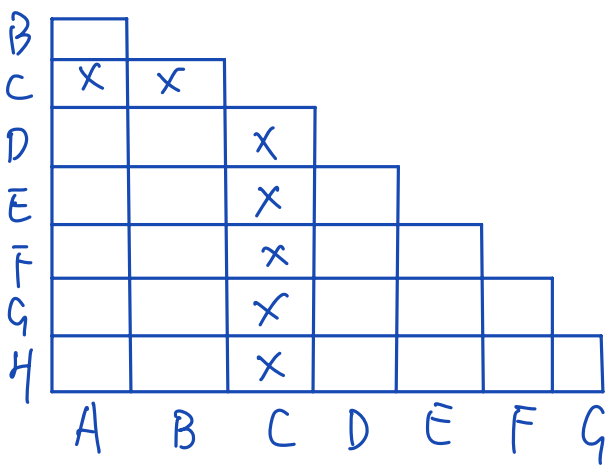
DFA 最小化：

DFA中两状态等价：$\tilde{\delta}(p,w) \in F \Leftrightarrow \tilde{\delta}(q,w) \in F$，否则可区分

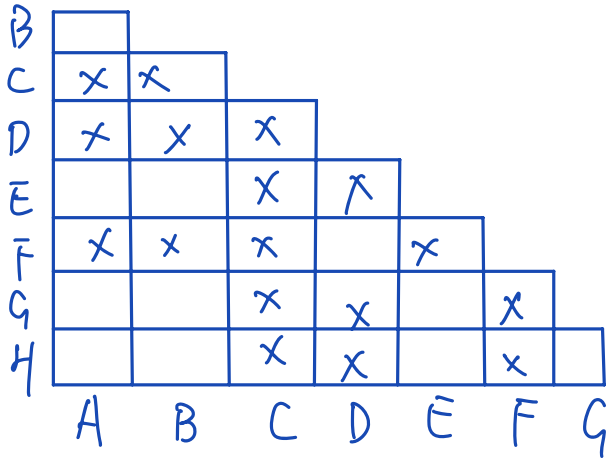
寻找可区分状态对：填表算法 (找可区分的状态对，打×)



① 接受状态和非接受状态可分

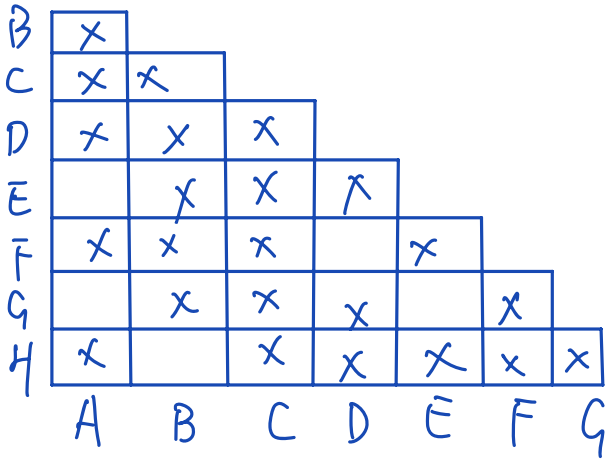


② 标记所有经过0到达终态和非终态的状态对



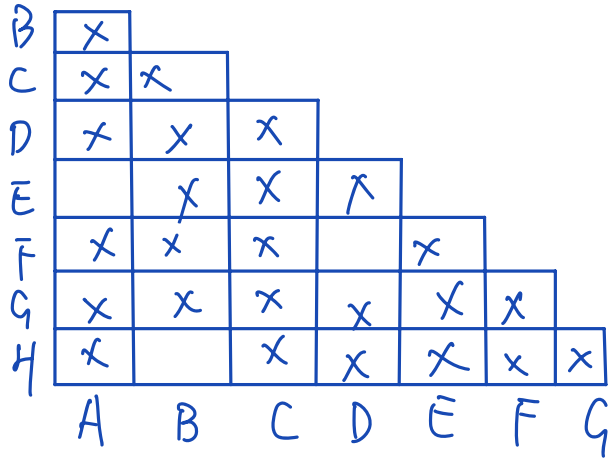
从D、F能到终态

③ 标记所有经过1到达终态和非终态的状态对



从B、H能到终态

④ 逐个检查剩余状态对



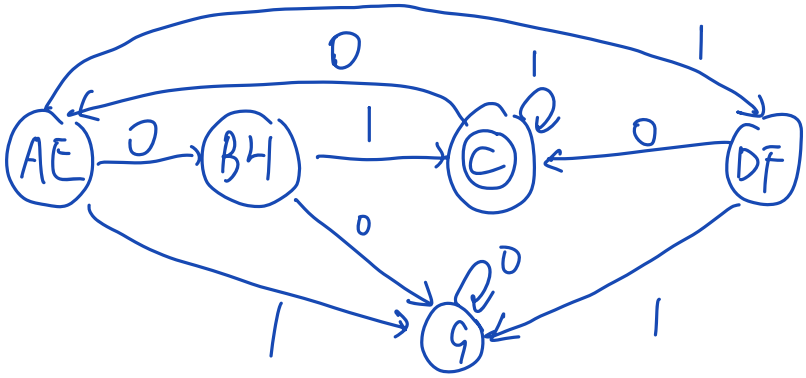
(A, E)：都能到C ✓

(A, G)：01→(C, E)

(B, H)：都能到C ✓

(D, F)：一样 ✓

(E, G)：10→(C, H)



可构造状态转移表

划分算法：① 从接受状态开始分，先分成 $G_1=F$，$G_2=G-F$

② 对于 G_2，画转移表，找出转移相同的分一组

③ 反复划分，直到最小

④ 将每组合并为一个状态，画DFA

词法分析 Lexical Analysis

词法分析：确定源程序w中每一个串 x 及其所属语言

记号(token)：(类别，值)　如 (NUM, 100)、(ID, x)、(INT, -)、(ADD, -)

类别：①ID：[a-z A-Z][a-z A-Z 0-9]*

②NUM：[+-]? [0-9]+

③CHR：字符　④FLO：.5 / -.23 / +31. / 3.14

⑤STR：字符串

⑥ADD：+　SUB：-　MUL：*　DIV：/　MOD：%　AAA：++　DEC：--

⑦LT：<　LE：<=　GT：>　GE：>=　EQ：==　NE：!=　AND：&&　OR：||　NOT：!

⑧BIT_AND/OR/NOT/XOR：&、|、~、^　SHL：<<　SHR：>>

⑨ASG：=　AAS：+=

⑩INT、IF、ELSE、WHILE、RETURN

⑪SCO(;)、CMA(,)、COLON(:)、LPA(()、RPA())、LBK([)、RBK(])

LBR({)、RBR(})

源程序 → (有效串+间隔)+ <EOF>　预处理：删空白、注释，处理宏

前缀最大化原则：从当前剩余输入串开始，尽可能取最长的合法串作为下一个词法单元。

事实优先级：同一个最长前缀属于多个语言，用优先级决定

↳ if $\in \{ID\}_{Keywords}$ → Keywords > ID

σ-DFA：多语言联合DFA（A，L，ψ）（A为DFA）同时识别多语言

L：语言簇　例：{ADD，AAS，ZNT，ZD，…}

ψ：ψ(q1)=ZD，ψ(q2)=NUM（接受状态到词法种属的映射）

构造：①为每个词法规则构造NFA，接受状态为该语言token

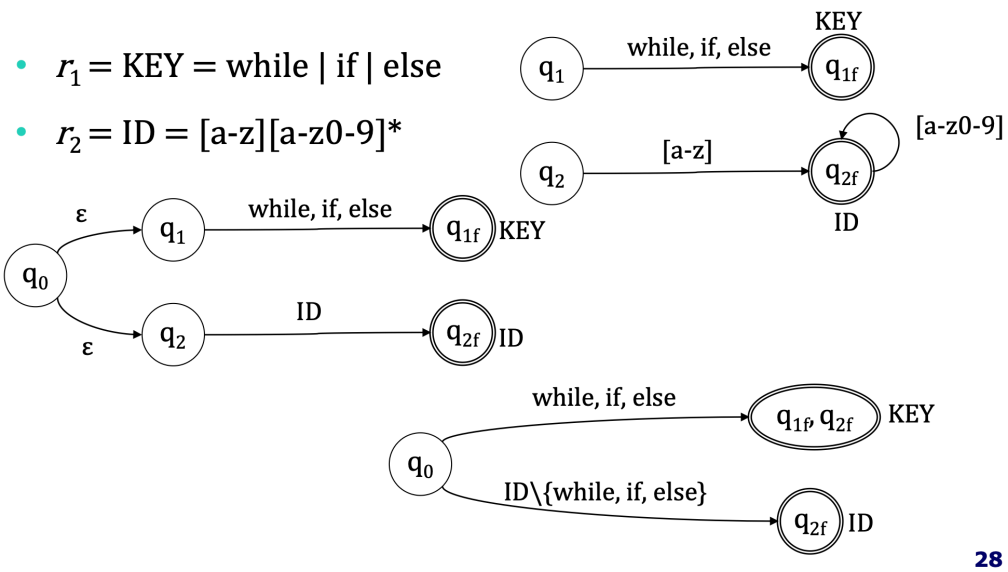
②加一个总初始状态q0，用ε连接每个NFA

③子集构造法，转为DFA

④给DFA接受状态标token（按优先级）

↓下标小，优先级高

- r_1 = KEY = while | if | else
- r_2 = ID = [a-z][a-z0-9]*



超前搜索：由于前缀最大化，读多了需要回退

上下文无关语言 CFG

CFG $G=(V,T,P,S)$ $\quad L(G)=L(S)\subseteq T^*$

↳ V：变元的有限集 (non-terminal)

T：终结符的有限集 (terminal)

P：产生式规则的有限集 (Production)

S：文法初始符号 (start symbol) $(S\in V)$

产生式：$A\to\alpha$ (α：候选式)

直接推导 (Derivation)：符号串 η 推导 δ：$\eta\Rightarrow\delta$ 把串里某变元换成候选式

推导 $\Rightarrow^*$：0到多个直接推导 (推导的闭包)

归约 (reduction)：推导的逆过程，把串换回变元，目标为S $\quad \alpha\gamma\beta\Leftarrow\alpha A\beta$

连续归约：$\Leftarrow^*$

句型：$S\Rightarrow^*\alpha$ 推出的任意串 (可含变元)

句子：$S\Rightarrow^* w$，全为终结符的串，$L(G)$=所有句子的集合

最左推导：$\Rightarrow_{lm}$，$\Rightarrow_{lm}^*$ 每步固定替换最左变元

最右推导：$\Leftarrow_{rm}$，$\Leftarrow_{rm}^*$ 每步固定替换最右变元

语法分析树：将推导写成树。根→初始符号，内节点→变元，叶子→终结符/ε。

产物：叶子从左到右连成的串

$\Leftarrow^*$，$\Rightarrow^*$，$\Rightarrow_{lm}^*$，$\Rightarrow_{rm}^*$，语法树等价

二义性文法：一个CFG的语言有元素至少是两个语法树的产物

句子有2棵语法树 ⇔ 有2个不同最左(右)推导

二义性是文法的性质，而非语言的。

二义性来源：① 运算符优先级 (+, *)　② 中缀运算符的结合性 (2+3+4)

③ 悬挂else：if e_1 then if e_2 then s_1 else s_2

固有二义性：有些CFL不存在无二义文法

例：$\{a^i b^j c^k \mid i=j \text{ or } j=k\}$

消除二义性：① 定结合性：$E \to E \oplus E$ 改为 { 左结合 $E \to E \oplus E \mid E'$; 右结合 $E \to E' \oplus E \mid E'$, $E' \to a$

② 分优先级：每个优先级配一个变元

$E \to E+T \mid T$, $T \to T*F \mid F$, $F \to (E) \mid a \mid i$

优先级高，靠近叶子，优先级低靠近根

左结合：堵死右边(等价的替换)，让左边递归；右结合相反

CFG化简：

(1) 去除无用符号：

有用：可达(从初始符号能推出) + 产出(能推出终结符串)

终结符总是产出，但不一定可达

先去掉无产出，再去掉不可达

(2) 去除ε表达式：$A \to \varepsilon$　　如果 $A \overset{*}{\Rightarrow} \varepsilon$，则A可空

变形表达式：对于一个产生式 $A \to \alpha$，α 中有 m 个可空，就把取ε和不取ε的符号情况进行组合

算法：① 求所有可空变元　② 每条非ε产生式的全部变形加入

③ 删掉ε

(3) 去除单位产生式：右边是单个变元的产生式

单位对(A,B): A经过一串单位产生式⇒B。

算法: ①求所有单位对

②对每个单位对(A,B),把B的所有非单位产生式加给A

③删掉单位产生式

顺序: 去ε → 去单位 → 去无产出 → 去不可达

例:

$S \to AB \mid aB \mid D$
$A \to aA \mid \varepsilon$
$B \to bB \mid b$
$C \to cc$
$D \to Dd$

去ε ⟹

$S \to AB \mid aB \mid D \mid B$
$A \to aA \mid a$
$B \to bB \mid b$
$C \to cc$
$D \to Dd$

去单位 ⟹

$S \to AB \mid aB \mid bB \mid b \mid Dd$
$A \to aA \mid a$
$B \to bB \mid b$
$C \to cc$
$D \to Dd$

去无用 ⟹

$S \to AB \mid aB \mid bB \mid b$
$A \to aA \mid a$
$B \to bB \mid b$

自上而下分析 LL(1)

问题：死循环(左递归)、回溯(匹配失效)

消除左递归：

(1) 直接左递归：$P \to P\alpha \mid \beta$

改为：$P \to \beta P' \quad P' \to \alpha P' \mid \varepsilon$

例：$T \to T*F \mid T/F \mid F$

$\Rightarrow T \to FT' \quad T' \to *FT' \mid /FT' \mid \varepsilon$

(2) 间接左递归：$N \to A\alpha \quad A \to N\beta \mid \gamma$

代入法，把 A 代入 N：$N \to N\beta\alpha \mid \gamma\alpha$

$\Rightarrow N \to \gamma\alpha N' \quad N' \to \beta\alpha N' \mid \varepsilon$

例：$S \to Qc \mid c \quad Q \to Rb \mid b \quad R \to Sa \mid a$

$\Rightarrow Q \to Sab \mid ab \mid b \quad R \to Sa \mid a$

$\Rightarrow S \to Sabc \mid abc \mid bc \mid c$

$\Rightarrow S \to abcS' \mid bcS' \mid cS' \quad S' \to abcS' \mid \varepsilon$

例：$A \to Bb \mid Ac \mid c \quad B \to Ba \mid Ad \mid d$

$\Rightarrow A \to BbA' \mid cA' \quad A' \to cA' \mid \varepsilon \quad B \to Ba \mid BbA'd \mid cA'd \mid d$

$\Rightarrow B \to cA'dB' \mid dB' \quad B' \to aB' \mid bA'dB' \mid \varepsilon$

消除回溯：

回溯来源：① 多个候选式有公共前缀 $A \to B+B \mid B-B$

② 候选式推出的串有终结符公共前缀

③ 有 ε 候选式带来的不确定

消除方法：① 让公共前缀只出现在单个产生式中

提取左因子(left factor)例：$N \to if\ then \mid if\ then\ else \Rightarrow N \to if\ then\ N' \quad N' \to else \mid \varepsilon$

① FIRST / FOLLOW.

FIRST(α)：α能推出的所有串的终结符前缀集合

$FIRST(\alpha)=\{a\in T \mid \alpha \Rightarrow^{*} a\beta\}$

规则：① First(t) = {t} (terminal)

② $\varepsilon \in First(X)$ { if $X\to\varepsilon$; if $X\to A_1A_2\cdots A_n$, $\varepsilon\in First(A_i)$ } （如果前面的可空，就把后面的FIRST加入）

③ $First(\alpha)\subseteq First(X)$, if $X\to A_1\cdots A_n\alpha$, $\varepsilon\in First(A_i)$

FOLLOW(A)：所有句型紧跟在A后面的终结符集合

规则：① $\# \in FOLLOW(A)$ (#为串的结尾标记)

② $FIRST(\beta)-\{\varepsilon\}\in FOLLOW(X)$, $A\to\alpha X\beta$

③ $FOLLOW(A)\subseteq FOLLOW(X)$, $A\to\alpha X\beta$ 且 $\varepsilon\in FIRST(\beta)$ （$A\to\alpha X$）

即 ① 开头符号加#

② $B\to\alpha A\beta$，接着A的β的FIRST(β)可作为A的FOLLOW，去掉ε后加入

③ $B\to\alpha A\beta$，但β可为空，A继承B的右邻居，即FOLLOW(B)给FOLLOW(A)

SELECT(α→): 决定当前输入符号选哪条候选式

$$SELECT(\alpha\to)=\begin{cases}(FIRST(\alpha\to)\setminus\{\varepsilon\})\cup FOLLOW(A) & ,\varepsilon\in FIRST(\alpha\to)\\ FIRST(\alpha\to) & ,\varepsilon\notin FIRST(\alpha\to)\end{cases}$$

LL(1)文法：同一变元候选式FIRST两两不相交；若某候选式FIRST含ε，还要求各候选式FIRST与左部FOLLOW不相交

改造：去ε(e)→去单位(u)→去无用(ag)→消左递归(r)→消回溯(c)→消二义(n)

判定是否LL(1)：每个变元的各候选式，SELECT集两两不相交，无左递归

如果可空，FIRST ∪ FOLLOW；不可空看FIRST。最后两两相交.

递归下降分析：每个变元对应一个递归过程，过程内按当前token落在哪个候选式的FIRST/FOLLOW来选候选式，遇到终结符则匹配，遇到变元调对应过程。

预测分析(表驱动)：

预测分析表 M[A,a]：

① $a \in FIRST(\alpha)$，$M[A,a]=\alpha$

② $\varepsilon \in FIRST(\alpha)$ 且 $a \in FOLLOW(A)$，则 $M[A,a]=\alpha$

画法：对每个文法，先求右侧的FIRST，填入对应位置。

如果FIRST有ε，看FOLLOW(A)的结果，在对应位置填ε

使用表分析：

栈	输入串	动作
E#	2-2*2#	E→TE'
TE'#	2-2*2#	T→FT'
FT'E'#		F→i
iT'E'#		match
T'E'#	-2*2#	T'→ε

注：栈反过来

二、 嵌套链表文法 $G(S)$ 定义如下：

$$S \rightarrow [L]$$
$$L \rightarrow L;L \mid S \mid a$$

其中：‘a’，‘[’，‘]’ 和 ‘;’ 为终结符，S 是**文法开始符号**. (25 分，每小题 5 分)

(1) 试写出语句“$[a;[a]]$”的一个最左推导；

(2) 试消除文法 $G(S)$ 中的左递归；

(3) 试对消除左递归后的文法所有非终结符求 First 集和 Follow 集；

(4) 试对消除左递归后的文法构造 LL(1) 分析表，从而说明消除左递归后的文法不是 LL(1) 文法；

(5) 试利用你的分析表写出语句“$[a;a]$”的一个正确的分析过程.

(1)

PDA

PDA $P=(Q,\Sigma,\Gamma,\delta,q_0,Z_0,F)$

Γ：栈符号的有限集

δ：转移函数 $Q\times(\Sigma\cup\{\varepsilon\})\times\Gamma\to 2^{Q\times\Gamma^*}$

Z_0：栈开始符号

PDA = NFA + 无穷栈　　弧标记 $a, X/\alpha$：读 a，弹出 X，压入 α

$(p,\gamma)\in\delta(q,a,X)$：在状态 q，读 a，栈顶 X 时，可转状态 p，并把栈顶换为 γ

瞬时描述 ID：(q,y,β)，q 状态，y 剩余串，β 栈内容（栈顶最左），初始 (q_0,w,Z_0)

直接移动 $(\vdash)$：若 $(p,\alpha)\in\delta(q,a,X)$，则 $(q,aw,X\beta)\vdash(p,w,\alpha\beta)$

↳ 读 a，弹 X，压 α

移动 $(\vdash^*)$：0 到多步直接移动。从初始 ID 到接受 ID 的移动路径叫成功路径。

性质：① 可加：若 $(q,x,\alpha)\vdash^*(p,y,\beta)$，则 $(q,xw,\alpha)\vdash^*(p,yw,\beta)$

若 $(q,x,\alpha)\vdash^*(p,y,\beta)$，则 $(q,x,\alpha\eta)\vdash^*(p,y,\beta\eta)$

合法 ID 剩余串末尾或栈底加，仍合法

② 可减：若 $(q,xz,\alpha)\vdash^*(p,yz,\beta)$，则 $(q,x,\alpha)\vdash^*(p,y,\beta)$

PDA 的语言：① 终结状态接受 $L(P)$：$(q_0,w,Z_0)\vdash^*(f,\varepsilon,\alpha)$，$f\in F$

② 空栈接受 $L(N)$：$(q_0,w,Z_0)\vdash^*(q,\varepsilon,\varepsilon)$

两语言等价

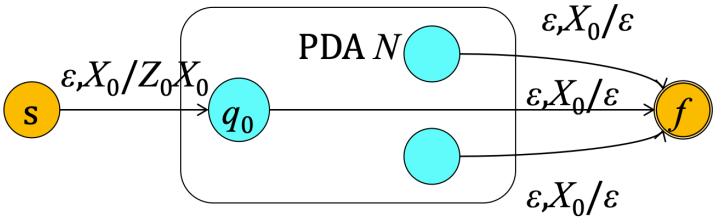
N：空栈　P：终态

N⇒P：① 保留N所有内容，额外加专用栈底标记X_0。

② 增加初始状态s和终结状态f

③ $\delta(s, \varepsilon, X_0) = \{(q_0, Z_0X_0)\}$

④ 对N每个状态，$\delta(q, \varepsilon, X_0) = \{(f, \varepsilon)\}$

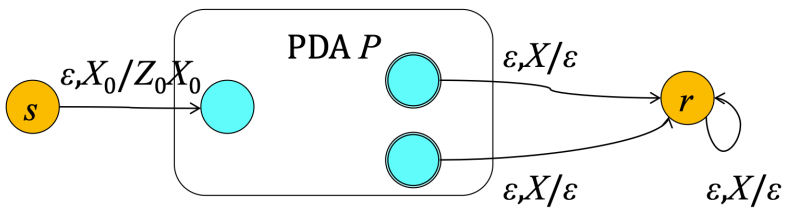


P⇒N：① 保留P内容，额外加专用栈底X_0

② 增加开始状态s和新状态r

③ $\delta(s, \varepsilon, X_0) = \{(q_0, Z_0X_1)\}$

④ 对P的每个终结状态f，$\delta(f, \varepsilon, X) = \delta(r, \varepsilon, X) = \{(r, \varepsilon)\}$



DPDA：任何情况下只有一条可走路径（如不能同时选读输入和ε移动）

DPDA语言：DPDA能模拟DFA（识别正则语言）

DPDA接受语言不是固有二义语言

DPDA严格弱于PDA，不等价

自下而上分析 (归约 $\Leftarrow^*$)

SRF：移进-归约框架

- 移进 shift：把当前输入符号压入栈顶
- 归约：当栈顶有可归约串(产生式右部)时，弹出换成左部变元

两类冲突：(根源在文法)

- 移进-归约冲突：栈顶有可归约的串，是否继续 shift
- 归约-归约冲突：栈顶可归约串可归约成多个不同变元

规范推导：最右推导　规范句型=右句型

规范归约($\mapsto$)：最右推导的逆过程。$\mapsto$可以是移进也可以是归约

短语：句型 $\delta A\beta$，若 $A \Rightarrow^+ \gamma$，则 γ 是句型 $\delta\gamma\beta$ 的短语

↳ 语法树里，任意一棵子树的产物

直接短语：$A \Rightarrow \gamma$　二层子树产物

句柄 handle：规范句型 δAx，最右推导为 $\delta\gamma x$，则 γ 为句柄

句柄是句型里最左边的直接短语

无二义文法，每个句型句柄唯一

规范归约每一步，就是把当前句柄换成其父变元

活前缀 viable prefix：规范句型的前缀，不包括句柄后的符号

最大活前缀：后缀正好是句柄

栈内容永远是活前缀，是最大活前缀就归约

(句柄出现在栈顶)

增广文法：加一个新初始符号：$S' \to S$

文法项目item：$A \to \alpha \cdot \beta$（α：已在栈上，读过；β：期待读到）

① $A \to \cdot \alpha\beta$：初始项目

② $A \to \alpha \cdot c\beta$：移进项目

③ $A \to \alpha \cdot N\beta$：待约项目

④ $A \to \alpha\beta \cdot$：归约项目（完全项目）

⑤ $S' \to S \cdot$：接受项目

构造item DFA（LR(0)规范簇）：

闭包closure(q)：项目集中有 $A \to \alpha \cdot N\beta$，就把 N 所有初始项目 $N \to \cdot\gamma$ 拉进这个集合，反复该过程。

算法：① 加入 $S' \to S$

② 对已有每个状态 q，每个文法符号 X：

找出所有类似 $A \to \alpha \cdot X\beta$ 的item，· 向右一步得 $A \to \alpha X \cdot \beta$，收集成一个集合，求其闭包，得到新状态

③ 重复 ② 直到不再产生新状态

④ 只含完全项目的状态是接受状态

LR(0)分析表：

(1) ACTION[m, a]（m状态，a终结符）：

- sn：item DFA 中有从 m 到 n，标记为终结符 a 的弧
- acc：m 含接受项目 $S' \to S$

• rk: m含编号k产生式的完全项目 (所有终结符列填rk)

(2) GOTO[m, A]: n → 从m到n, 权记为变元A的弧

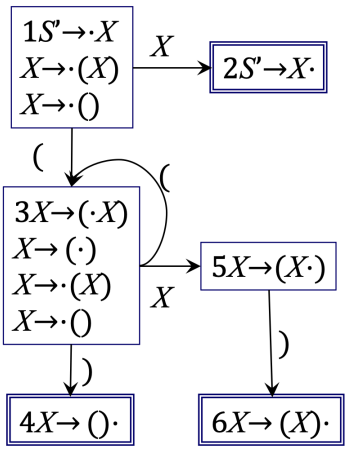
双线分析算法: 状态栈, 符号栈

初始状态栈放S', 符号栈放#, 查ACTION[栈顶状态, 当前输入]:

① sn: 把输入符号压符号栈, n压状态栈, 读下一个输入符号

② rk(A→α): 两栈各弹出|α|个, 看新栈顶m', A压符号栈, GOTO[m', A]压状态栈

③ acc→成功, 空→报错



	ACTION			GOTO
状态	(	)	#	X
1	s3			2
2			acc	
3	s3	s4		5
4	r3	r3	r3	
5		s6		
6	r2	r2	r2	

ACTION[m,a]:

- sn: bi-push(n, a)
- rn: 按产生式nA→α归约: bi-pop(|α|);bi-top(m', X] bi-push((GOTO[m', A], A)。
- acc: 分析成功。

状态栈	符号栈	剩余串	动作
1	#	(())#	shift
13	#(	())#	shift
133	#((	))#	shift
1334	#()	)#	reduce X→()
135	#(X	)#	shift
1356	#(X)	#	reduce X→(X)
12	#X	#	acc

1S'→X
2X→(X)
3X→()

SLR(1): 多看一个输入, 用FOLLOW来决定:

① 含完全项目A→β·, 且输入a ∈FOLLOW(A), 用A→β归约

② 含移进项目A→α·cβ, 且输入a=c, 移进

③ 否则报错

SLR(1)条件:

某状态含m个移进项目(·C), n个完全项目A_i→α·, 满足:

① C与每个FOLLOW(A_i)都不相交

② 任意两个完全项目的FOLLOW(A_i), FOLLOW(A_j)不相交

↳保证每个输入符号最多触发一个动作

SLR(1)分析时消除s-r冲突

左结合归约，右结合移进

任何二义文法都不是LR文法

实践中保留二义文法+额外消解规则：

①运算符优先级+结合性：优先级高的先shift/reduce

②悬挂else：最近匹配原则，状态遇到else移进，遇到#归约

③并列符号($S \to SS$)：左/右结合规则消解

④修剪文法，延迟归约

语义分析

任务：①类型检查　②作用域与标识符检查(声明、重定义、作用域)

③控制流检查(语句位置、返回值)

④收集语义信息(类型、地址)

合法性：语义规则满足　正确性：程序逻辑对

符号表：汇集源程序里各名字的属性，表头+登记项

表头：①outer：外层符号表指针

②width：函数存储区大小

③argc：参数个数

④arglist：参数名列表

⑤rtype：返回结果类型

⑥level：嵌套层数

⑦code：中间代码

⑧entry：登记项链

登记项：　固定：①name：名字

②type：类型(INT, ARRAY, FUNC)

特殊：简单变量：③offset：相对地址

数组：③etype：元素类型

④base：首址

⑤dims：维数(几个[])

⑥dim[]：各维维长([]内数)

函数：③ mytab：指向该函数自己的符号表

④ offset：偏移

操作函数：① bind(x, type)：给当前符号表加登记项（已存在则报错）

② lookup(x)：在当前符号表或作用域链中查找 x

lookup(x, field:)：返回 x 登记项中的 field 字段

lookup(x, field: value)：修改 x 登记项中的 field 字段

③ update[field, value, op]：修改符号表表头字段

④ bind1(tab, …), lookup1(tab, …)：对指定表操作

当前符号表：全局栈 symtab 的栈顶，进入一个函数，压入一个新表，结束后弹出

层次符号表：outer 串嵌套关系，level 记层数

属性文法：att(G) = <G, E>，给文法 G 的每条产生式配属性方程

例：$S \to SS$ {

S[0].level = max(S[1].level, S[2].level)

}

$S \to (S)$ { S[0].level = S[1].level + 1 }

$S \to ()$ { S.level = 1 }

属性名：如 val, type, place, level

属性类：产生式中每个文法符号

属性值：实例的取值

同一变元多次出现用 [0], [1], [2] 区分

综合属性：父结点属性由子结点属性计算得到，适合自底向上

$E \to E + E \ \{E[0].val = E[1].val + E[2].val\}$

继承属性：子结点属性由父结点或左兄弟传下来，适合自顶向下

$D \to TV \ \{V.in = T.type\}$

属性求值：(1) 依赖图拓扑排序：属性A依赖B（A→B）

依赖图必须无环　　拓扑排序得次序

(2) 遍历语法树：综合属性 → 后根遍历（先子后父）

继承属性 → 先根遍历（先父后子）

S-属性文法：只有综合属性，适合SLR

L-属性文法：任一继承属性不依赖其右兄弟，适合LL

语法制导翻译：属性求值，产生代码

S-属性文法的LR制导求值：状态栈、符号栈、每个属性的属性栈

S'→S　{S'.l=S.l; S'.m=S.m}

S→SS　{S[0].l=maS(S[1].l,S[2].l);　S[0].m=S[1].m+S[2].m}

S→(S)　{S[0].l=S[1].l+1;　S[0].m=S[1].m+1;}

S→()　{S.l=S.m=1}

符号栈	l栈	m栈	剩余串	动作
#	#	#	()(())#	移进
#(	#-	#-	)(())#	移进
#()	#--	#--	(())#	r4; S.l=1; S.m=1
#S	#1	#1	(())#	移进
#S(	#1-	#1-	())#	移进
#S((	#1--	#1--	))#	移进
#S(()	#1---	#1---	)#	r4; S.l=1; S.m=1
#S(S	#1-1	#1-1	)#	移进
#S(S)	#1-1-	#1-1-	#	r3; S[0].l=S[1].l+1; S[0].m=S[1].m+1
#SS	#12	#12	#	r2; S[0].l=max(S[1].l,S[2].l); S[0].m=S[1].m+S[2].m
#S	#2	#3	#	acc

移进时属性压入属性栈

归约时属性栈弹出右部变元属性，压入左部属性

S-属性制导处理不了继承属性：S只自底向上，提前归约

$D \to T\tilde{V}$ { $\tilde{V}$.in = T.type }　　int x,y,z; 归约x时，不知type

$\tilde{V} \to d$ { bind(getn(d), $\tilde{V}$.in) }

解决：把继承属性写为综合属性，先收集待处理的名字，归约时统一bind：

$D \to T\tilde{V}$ { for(e∈$\tilde{V}$.in) bind(e, T.type) }

$\tilde{V} \to \tilde{V}_1, d$ { $\tilde{V}$[0].in = cons(getn(d), $\tilde{V}$[1].in) }

$\tilde{V} \to d$ { $\tilde{V}$.in = list(getn(d)) }

相关函数：①getn(d)：get name，取d的名字

②getv(i)：get value，取值

③list(x)：建表

④cons(x,l)：头插表

⑤endcons(l,x)：尾插表

⑥for(e←list)：遍历表

⑦newvar()：创建并返回一个变量名

程序声明的语义分析：

(1) 简单变量声明：

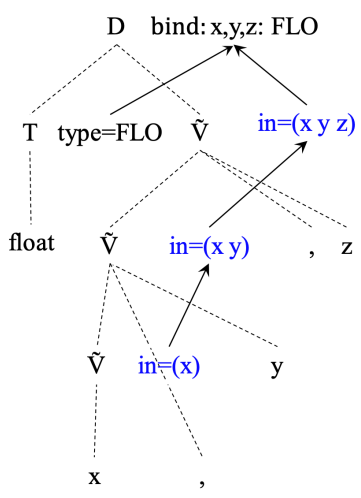
D→T$\tilde{V}$ {
 for(e∈$\tilde{V}$.in)bind(e, T.type);
 D.place=$\tilde{V}$.in}

T→int {T.type=INT}

T→float {T.type=FLO}

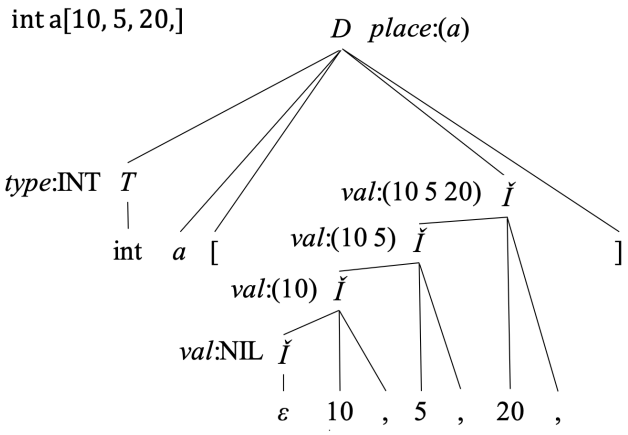
$\tilde{V}$→$\tilde{V}$,d {v=getn(d);
 $\tilde{V}$[0].in=endcons($\tilde{V}$[1].in), v)}

$\tilde{V}$→d {v=getn(d);
 $\tilde{V}$.in=list(v))}



float x, y, z

(2) 数组变量声明: D → Td[I]



width+=4000
bind(*a*, ARRAY)
base:width
etype:INT
dims:3
dim[0]:10
dim[1]:5
dim[2]:20

- int a[10,5,20]
- 数组登记项内容为，
  - name:a;
  - type:ARRAY;
  - base:有; // tab->width
  - etype:INT;
  - dims:3;
  - dim[0]:10;
  - dim[1]:5;
  - dim[2]:20
- tab->width增加4000

(3) 函数声明:

例:

```
@table: (
outer: NULL
width: 428 argc: 0 arglist: NIL rtype: VOID level: 0 code: [...]
entry: (name: a type: ARRAY base: 400 dims: 1 dim[0]: 100 etype: INT)
entry: (name: x type: INT offset: 404)
entry: (name: reada type: FUNC offset: 412 mytab: reada@table)
entry: (name: pivot type: FUNC offset: 420 mytab: pivot@table)
entry: (name: qsort type: FUNC offset: 428 mytab: qsort@table))

reada@table: (
outer: @table
width: 4 argc: 0 arglist: NIL rtype: VOID level: 1 code: [...]
entry: (name: i type: INT offset: 4))

pivot@table: (
outer: @table
width: 8 argc: 2 arglist: (low high) rtype: INT level: 1 code: [...]
entry: (name: low type: INT offset: 4)
entry: (name: high type: INT offset: 8))
```

```
int z;
int a[10,20];
float foo(int x, int b[], int boo()){
        int bar(int x){
                return x++};
        if (x >0) z=sqrt(b[0,0])
        else return boo(z)};
print foo(0, a[],bar())
```

@table: (
outer: NIL
width: 812
argc: 0
arglist: NIL
rtype: INT
code: @code([...])
entry: (
name: z
type: int
offset: 4
)
entry: (
name: a
type: ARRAY
etype: INT
base: 804
dims: 2
dim[0]: 10
dim[1]: 20
)
entry: (
name: foo
type: FUNC
offset: 812
mytab: foo@table
)
)

```
foo@table : (
    outer : @table
    width : 24
    argc : 3
    arglist : (x  b  boo)
    rtype : FLO
    code : foo@code(...)
    entry : (
        name : x
        type : INT
        offset : 4
    )
    entry : (
        name : b
        type : ARRPTT
        offset : 8
        etype : INT
    )
    entry : (
        name : boo
        type : FUNPTT
        offset : 16
        rtype : INT
    )
    entry : (
        name : bar
        type : FUNC
        offset : 24
        mytab : bar@table
    )
)
```

```
foo.bar@table : (
    outer : @table
    width : 4
    argc : 1
    arglist : (x)
    rtype : INT
    code : bar@code
    entry : (
        name : x
        type : INT
        offset : 4
    )
)
```

中间代码生成 —— 三地址码 (TAC)

例：

```
int x;
int fact(int n; int a;){
    if (n<=1) return a
    else return fact (n-1, n*a,)
};
x=123+fact(5,1,);
print x
```

```
@code=[
    t4=123 ; t5=5; t6=1;
    PAR t6; PAR t5;
    t7= CALL fact,2;
    x= t4+ t7;
    PRINT x]
```

```
fact@ code=[
    IF n==1 THEN l1 ELSE l2;
    LABEL l1; RETURN a;
    GOTO l3;
    LABEL l2;
    t1=n-1; t2=n*a;
    PAR t2; PAR t1;
    t3= CALL fact,2;
    RETURN t3;
    LABEL l3
]
```

常用 TAC：LABEL l ; GOTO e; IF q op a THEN l1 ELSE l2

PAR a ; q=CALL d,k ; RETURN a ; q=CONVERT q,t

q=M[q] ; M[q]=a

TAC → 四元式：$q_2 = q_1$ aop a ⟶ (aop, q_1, a, q_2)

IF x<y THEN l1 ELSE l2 ⟶ (J<, x, y, l1) (J, _, _, l2)

t = M[addr] ⟶ (LOAD, addr, _, t)

M[addr] = x ⟶ (STORE, x, _, addr)

重要属性：

E.place：表达式结果所在位置

E.code：计算表达式所需代码

B.tc：布尔表达式为真时跳转的符号列表

B.fc：布尔表达式为假时跳转的符号列表

B.code：判断布尔表达式并跳转的代码

S.code：语句生成的代码块

$\check{E}$.place：表达式列表中每个表达式结果位置组成的列表

$\check{E}$.code：表达式列表中每个代码组成的列表

$\check{R}$.place: 实参列表中每个实参结果位置组成的列表

$\check{R}$.code: 实参列表中每个实参代码组成的列表

简单算术表达式翻译:

例: $a * (x-2) + y$

$t_1 = 2$; $t_2 = x-2$; $t_3 = a * t_2$; $t_4 = t_3 + y$

E.place = t_4

表达式列表和整数列表:

例: 1+2, x, y*z

$\check{E}$.place = (t_3, x, t_4)

$\check{E}$.code = ([$t_1 = 1$; $t_2 = 2$; $t_3 = t_1 + t_2$],

[],

[$t_4 = y * z$])

[10, 5, 20]

$\check{I}$.val = (10, 5, 20)

常量折叠: 一部分全是常数, 可以直接算

赋值: 例: x = y+1;

$t_1 = 1$; $t_2 = y + t_1$; x = t_2

布尔表达式: $B \rightarrow B \wedge B$ {

B[0].t_L = B[2].t_L;

B[0].f_L = append (B.[1].f_L, B[2].f_L);

B[0].code = B[1].code

++ LABEL B[1].tc

++ B[2].code

}

B → B V B {

B[0].tc = append(B[1].tc, B[2].tc);

B[0].fc = B[2].fc;

B[0].code = B[1].code

++ LABEL B[1].fc

++ B[2].code

}

B → !B {

B[0].tc = B[1].fc;

B[0].fc = B[1].tc;

B[0].code = B[1].code;

}

控制语句: if (B) S: B.code

LABEL B.tc

S.code

LABEL B.fc

if (B) S else S :

B.code

LABEL B.tc

S1.code

GOTO next

LABEL B.fc

S2.code

LABEL next

while (B) Body :

LABEL begin

B.code

LABEL B.tc

Body.code

GOTO begin

LABEL B.fc

例: if (123 ∧ x < 1) while (z) { x=1; y=b }

QTAC:

$t_1 = 123;$

IF $t_1 != 0$ THEN l_1 ELSE l_2;

LABEL l_1;

$t_2 = 1$;

IF $x < t_2$ THEN l_3 ELSE l_4;

LABEL l_3;

LABEL begin;

IF z!=0 THEN L5 ELSE L6;

LABEL L5;

t_3=1 ; x= t_3; y= b;

GOTO begin;

LABEL L6;

LABEL L2;

LABEL L4;

数组: 例: int a[2,3];

a[k-1, 1] = 5;

QTAC: t_1= 1 ; t_2= k-t_1;

t_3= t_2 * 3 ; t_4= t_3 + t_1;

t_5= t_4*4; t_6= a+ t_5;

M[t_6]= 5;

即 a[i,j] ⟶ a+ (i * dim[1]+j) * sizeof (etype)

函数调用: 先算实参, 倒序PAR, 再CALL

例: fact(n-1, n*a,)

QTAC: t_1=n-1 ; t_2=n*a;

PAR t_2; PAR t_1;

t_3= CALL fact, 2;

运行时环境

指令选择：① 0-DFA指令选择（贪心）

② BURS：树模式匹配+动态规划

指令合并：$t=r_s+k$ $\Longrightarrow$ lw r_t, k(r_s)

$r_t=M[t^{last}]$

IF a=c THEN l_1 ELSE l_2; $\Rightarrow$ beq a, c, l_1

LABEL l_2; l_2:

函数序言：建立栈帧　　尾声：恢复调用者环境

可执行程序：可执行文件（磁盘，静态，无栈帧）

内存映像（内存，动态，加载时分配堆栈）

运行时区域：↓ 代码区

静态区（全局/静态）

堆区（动态）

↑ 栈区（局部/栈帧）

名字的运行时环境：中间语言代码中出现的名字，都分配在它们声明宿主的最新栈帧里

栈帧：



sp指向栈顶

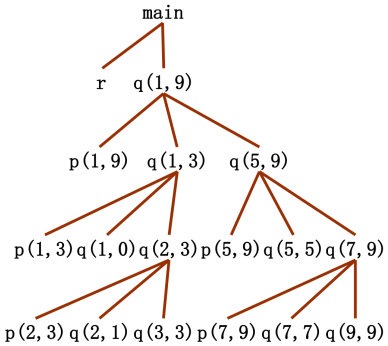
fp指向栈帧对象（控制链）

sp和fp指向同一栈帧，其为当前栈帧

[0]：代码入口@lable　[1]：访问链/环境指针

活动：过程的一次执行

活动树：描述活动嵌套关系



访问链：静态外层活动记录（谁声明）

控制链：返回时恢复调用者活动记录（谁调用）

参数传递：（默认传值）

① 传值：把实参复制给形参

② 传地址：把实参地址传给形参，形参是实参别名

③ 得结果：调用传值，出口把形参值拷回实参

④ 传名：类宏替换，把函数内所有形参名字都改实参